

Interface in C# with Examples

In this article, I will discuss one of the most important concepts, i.e., **Interface in C#**, with Examples. Please read our previous article discussing [Abstract Classes and Methods in C#](#) with Examples. At the end of this article, you will understand the following pointers.

1. [What is an Interface in C#?](#)
2. [Differences Between Concrete Class, Abstract Class, and Interface in C#](#)
3. [How to Define an Interface in C#?](#)
4. [How to Define Abstract Methods in an Interface in C#?](#)
5. [What are the Members we can and can't define in an interface in C#?](#)
6. [Multiple Examples to Understand Interface in C#](#)
7. [Explicit Interface Implementation in C#](#)
8. [When to use Interface in C#?](#)

What is an Interface in C#?

The Interface in C# is a **Fully Unimplemented Class** used for declaring a set of operations/methods of an object. So, we can define an interface as a pure abstract class, which allows us to define only abstract methods. The abstract method means a method without a body or implementation. It is used to achieve multiple inheritances, which the class can't achieve. It is used to achieve full abstraction because it cannot have a method body.

In C#, an interface is a fundamental concept defining a contract or a set of rules a class must adhere to. It specifies a list of methods, properties, events, or indexers a class implementing the interface must provide. Interfaces allow you to define a common set of functionality that multiple classes can share, promoting code reusability and ensuring a consistent structure for related classes.

Till now, we are learning classes. So, what is a class? A class is a user-defined data type. Then, what is an interface? An interface is also a user-defined data type. Then what is the difference between them? Let us understand this first.

Differences Between Concrete Class, Abstract Class, and Interface in C#:

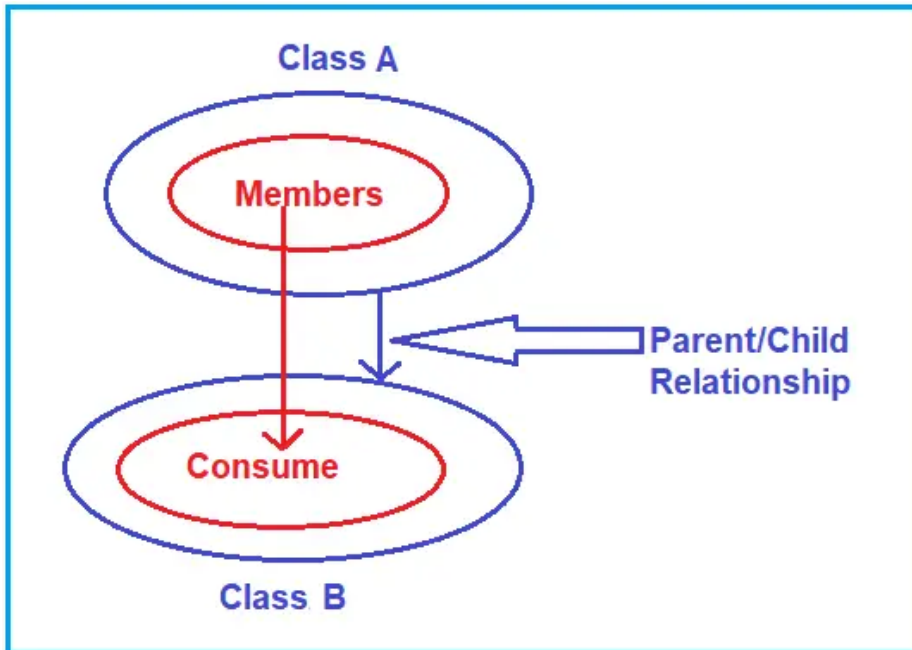
A Concrete Class can contain only the methods that contain the method body. Do we have a method without the body? Yes, we have methods without a body, called abstract methods. So, we can say class contains a method with a method body, or you can say non-abstract methods. Abstract class contains abstract and non-abstract methods, and the interface contains only abstract methods.

1. **Class**: Contains only the Non-Abstract Methods (Methods with Method Body).
2. **Abstract Class**: Contains both Non-Abstract Methods (Methods with Method Body) and Abstract Methods (Methods without Method Body).
3. **Interface**: Contain only Abstract Methods (Methods without Method Body).

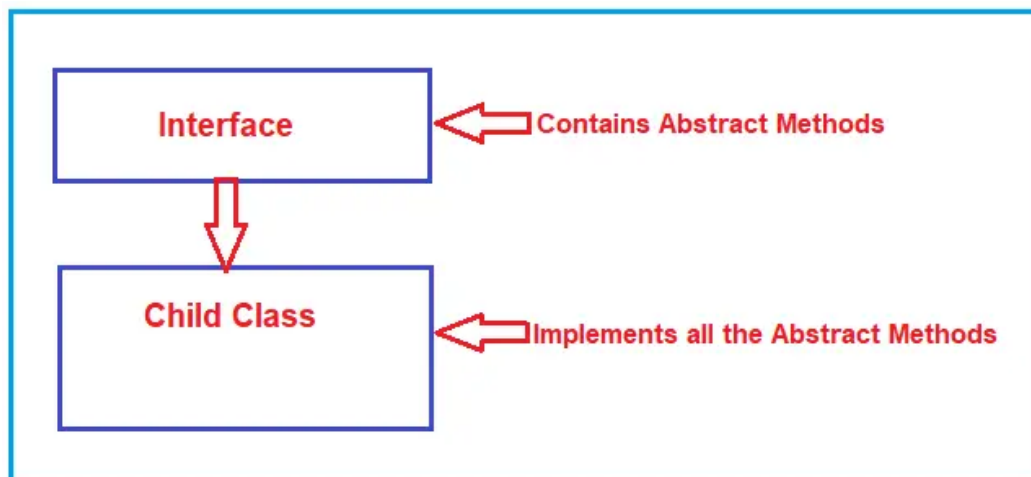
What are abstract methods, why do we need abstract methods, and how do we implement abstract methods? We have already discussed this in our [Abstract Classes and Abstract Methods in the C#](#) article.

Note: Every abstract method of an interface should be implemented by the child class of the interface without fail (Mandatory).

In **Inheritance**, we already learned that a class inherits from another class, and the child class consumes the Parent class members. Please observe the following diagram. Here, we have class A with a set of members, and class B inherits from class A. And there is a relationship called Parent/Child relation between them. Once the Parent/Child relationship is established, then the members of class A can be consumed under class B. So, this is what we learned in Inheritance.



Now, just like a class has another class as a Parent, a class can also have an Interface as a Parent. If a class has an interface as a Parent, the class is responsible for implementing all the abstract methods of the interface. For a better understanding, please have a look at the below diagram.



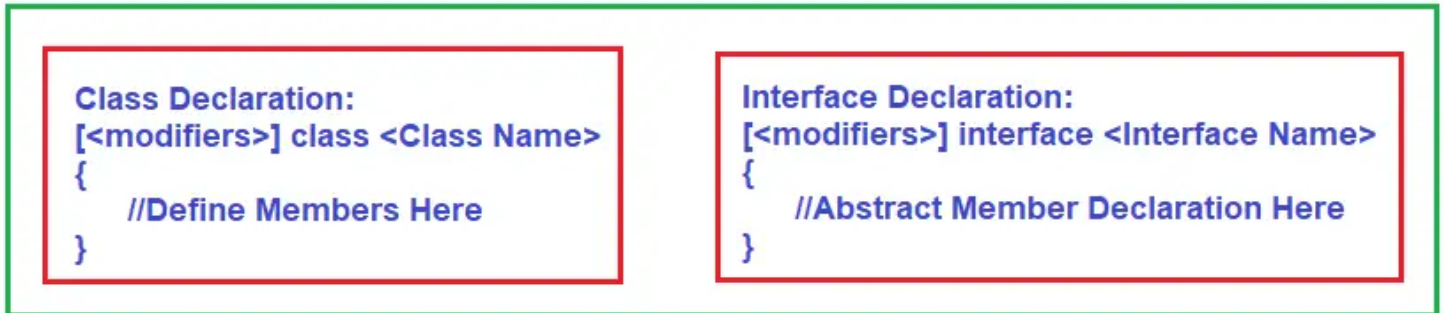
Simply speaking, the Parent Interface imposes restrictions on the Child Classes. **What restrictions?** The restrictions are to implement each and every method of the interface under the child class.

Generally, a class inherits from another class to consume members of its Parent. On the other hand, if a class inherits from an interface, it is to implement the members of its Parent (Interface), not for consumption.

Note: A class can inherit from a class and interface(s) at a time.

How to Define an Interface in C#?

The way we define a class in the same way we need to define an interface in C#. In a class declaration, we need to use the class keyword, whereas in an interface declaration, we need to use the interface keyword. Moreover, in an interface, we can only declare abstract members. For a better understanding, please have a look at the below diagram.



For a better understanding, please have a look at the below example. Here, we have created one interface with the name `ITestInterface` by using the interface keyword.

```
interface ITestInterface
{
    //Abstract Member Declarations
}
```

How to Define Abstract Methods in an Interface in C#?

In a class (i.e., Abstract Class), we generally use the abstract keyword to define abstract methods as follows.
`public abstract void Add(int num1, int num2);`

If you want to write the above abstract method in an interface, then you don't require public and abstract keywords in the method signature as follows:

```
void Add(int num1, int num2);
```

While working with Interface, we need to remember some Rules. Let us discuss those rules one by one with Examples.

Point 1: The first point that you need to remember is that the default scope for an interface's members is public, whereas it is private in the case of a class.

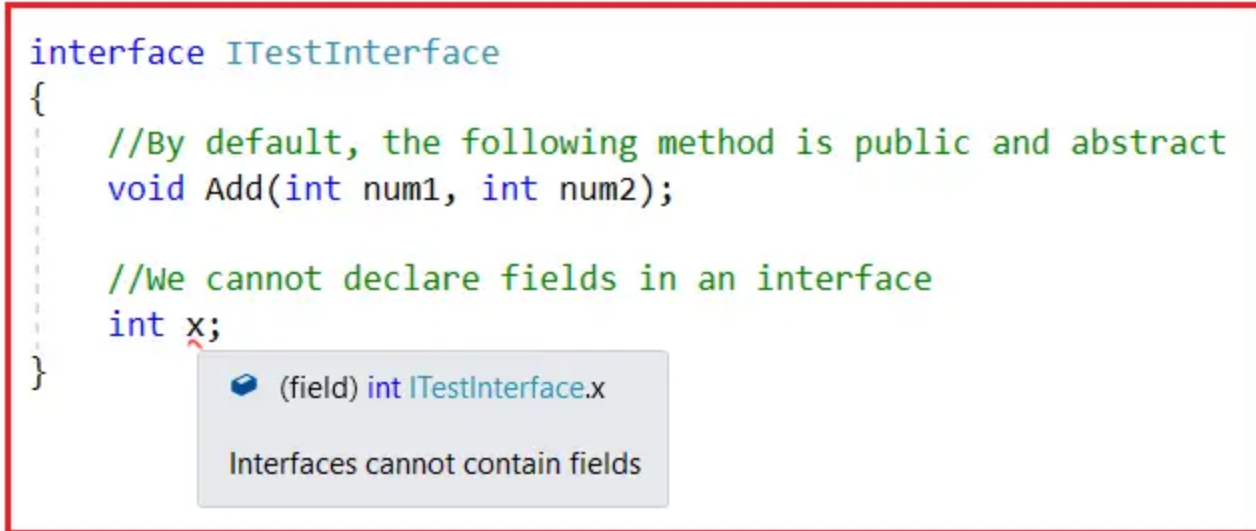
Point 2: The second point that you need to remember is by default, every member of an interface is abstract, so we aren't required to use the abstract modifier on it again, just like we do in the case of an abstract class. For a better understanding, please have a look at the below example. By default, the Add method is going to be public and abstract.

```
interface ITestInterface
{
    //By default, the following method is public and abstract
```

```
    void Add(int num1, int num2);  
}
```

Point 3: You need to remember that we cannot declare fields/variables, constructors, and destructors in an interface in C#.

Please have a look at the below code. Here, we are trying to declare a variable, and as soon as we declare, we get one compile-time error, i.e., **Interfaces cannot contain fields** as shown in the below image.



What are the Members we can and can't define in an interface in C#?

An interface can contain

1. **Abstract methods**
2. **Properties**
3. **Indexes**
4. **Events**

An interface cannot contain

1. **Non-abstract functions**
2. **Data fields**
3. **Constructors**
4. **Destructors**

Point 4: The fourth point you need to remember is that an interface can inherit from another interface in C# just like a class inherits from another.

For a better understanding, please have a look at the below code. Here, we have two interfaces, i.e., Interface1 and Interface2. Interface2 is inherited from Interface1, and now interface2 has two abstract methods, i.e., Add (from Interface 1) and Sub.

```

interface ITestInterface1
{
    //By default, the following method is public and abstract
    void Add(int num1, int num2);
}
interface ITestInterface2 : ITestInterface1
{
    //By default, the following method is public and abstract
    void Sub(int num1, int num2);
}

```

Now, the Child class that inherits from Interface1 has to implement one method, i.e., Add, and the child class that inherits from Interface2 has to implement two methods, i.e., Add and Sub.

You may have one question: **why do we need two separate interfaces? Why not one?** Yes, you can use one interface and define all the methods as long as they relate to one task. If you put non-related methods in a single interface, then it is against the SOLID – Interface Segregation Principle. If you want to learn the **SOLID – Interface Segregation Principle**, please click [here](#).

Point 5: The fifth point that you need to remember is every member of an interface should be implemented under the child class without fail (mandatory), but while implementing, we aren't required to use the override modifier just like we have done in the case of an abstract class.

For a better understanding, please have a look at the following code. Here, we have two interfaces and two implementation classes. Interface2 is inherited from Interface1, and hence, it has two abstract methods. ImplementationClass1 inherits from Interface1 and hence implements the Add method only. ImplementationClass2 inherits from Interface1, and Interface2 inherits from Interface1, and hence, this class needs to implement both abstract methods. That is what you can see in the below code.

```

interface ITestInterface1
{
    void Add(int num1, int num2);
}
interface ITestInterface2 : ITestInterface1
{
    void Sub(int num1, int num2);
}
public class ImplementationClass1 : ITestInterface1
{
    //Implement only the Add method
    public void Add(int num1, int num2)
    {
        Console.WriteLine($"Sum of {num1} and {num2} is {num1 + num2}");
    }
}
public class ImplementationClass2 : ITestInterface2
{
    //Implement Both Add and Sub method

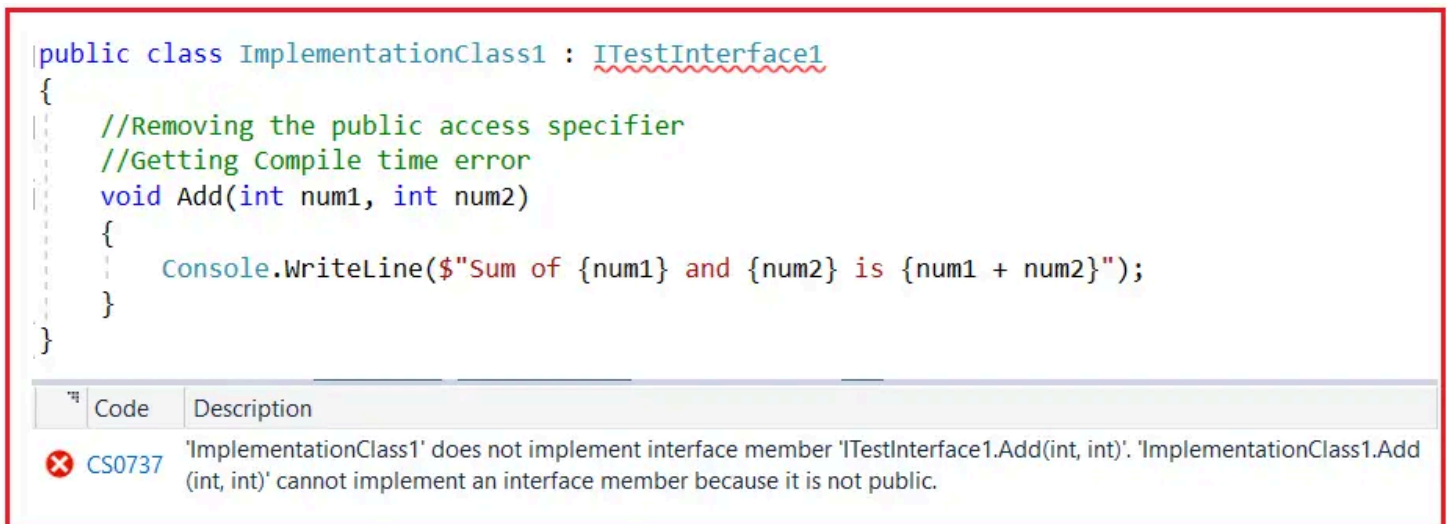
```

```

public void Add(int num1, int num2)
{
    Console.WriteLine($"Sum of {num1} and {num2} is {num1 + num2}");
}
public void Sub(int num1, int num2)
{
    Console.WriteLine($"Divison of {num1} and {num2} is {num1 - num2}");
}
}

```

In the above example, you can see that while implementing the method, we are using the public modifier, which is required. If you don't use public, it will treat the method as private, and you will get a compiler error: **'ImplementationClass1' does not implement interface member 'ITestInterface1.Add(int, int)'. 'ImplementationClass1.Add(int, int)' cannot implement an interface member because it is not public.** as shown in the below image.



Example to Understand Interface in C#:

Whatever we discussed as of now, we have put all these things in the below example. Please go through the comment lines.

```

using System;
namespace AbstractClassMethods
{
    class Program
    {
        static void Main()
        {
            ImplementationClass1 obj1 = new ImplementationClass1();
            //Using obj1 we can only call Add method
            obj1.Add(10, 20);
            //We cannot call Sub method
            //obj1.Sub(100, 20);
            ImplementationClass2 obj2 = new ImplementationClass2();
            //Using obj2 we can call both Add and Sub method

```

```

        obj2.Add(10, 20);
        obj2.Sub(100, 20);
        Console.ReadKey();
    }
}

interface ITestInterface1
{
    void Add(int num1, int num2);
}
interface ITestInterface2 : ITestInterface1
{
    void Sub(int num1, int num2);
}
public class ImplementationClass1 : ITestInterface1
{
    //Implement only the Add method
    public void Add(int num1, int num2)
    {
        Console.WriteLine($"Sum of {num1} and {num2} is {num1 + num2}");
    }
}
public class ImplementationClass2 : ITestInterface2
{
    //Implement Both Add and Sub method
    public void Add(int num1, int num2)
    {
        Console.WriteLine($"Sum of {num1} and {num2} is {num1 + num2}");
    }
    public void Sub(int num1, int num2)
    {
        Console.WriteLine($"Divison of {num1} and {num2} is {num1 - num2}");
    }
}
}

```

Output:

```

Sum of 10 and 20 is 30
Sum of 10 and 20 is 30
Divison of 100 and 20 is 80

```

Point 6: We cannot create an instance of an interface, but we can create a reference of an interface. The interface reference is going to hold the child class instance. We can only invoke the methods declared in the interface using the interface reference.

For a better understanding, please have a look at the below example. In the below example, ITestInterface1 declared one abstract method, i.e., Add. This interface is then implemented by the ImplementationClass, which implements the Add interface method. Again, we defined a new method in this class, i.e., Sub. Next, inside the Main method, we create a reference of the interface pointing to the child class instance. Using this

reference, we can only invoke the Add method, and we cannot invoke the Sub method. This is because the Add method signature is inside the interface, but the Sub method signature is not in the interface.

Example to Understand Interface Reference in C#:

```
using System;
namespace AbstractClassMethods
{
    class Program
    {
        static void Main()
        {
            //Creating Reference of an Interface point to the
            //child class instance
            ITestInterface1 obj = new ImplementationClass();
            //Add method signature declared in ITestInterface1, so we can
            //Invoke the Add method
            obj.Add(10, 20);
            //Sub method signature is not declared in ITestInterface1,
            //so, we cannot Invoke the Sub method
            //obj.Sub(100, 20);

            Console.ReadKey();
        }
    }

    interface ITestInterface1
    {
        void Add(int num1, int num2);
    }

    public class ImplementationClass : ITestInterface1
    {
        //Interface Method Implementation
        public void Add(int num1, int num2)
        {
            Console.WriteLine($"Sum of {num1} and {num2} is {num1 + num2}");
        }
        //This method purely belongs to ImplementationClass
        public void Sub(int num1, int num2)
        {
            Console.WriteLine($"Divison of {num1} and {num2} is {num1 - num2}");
        }
    }
}
```

Explicit Interface Implementation in C#

When each interface method is implemented separately under the child class by providing the method name and the interface name, it is called Explicit Interface Implementation. But in this case, while calling the

method, we should compulsorily use the interface reference created using the object of a class or typecast the object to the appropriate interface type.

You can also implement an interface another way without using the public access modifier. In this case, we need to specify the interface name before the method name using the dot operator, as shown in the below code. This is called Explicit Implementation of Interface Methods.

```
interface ITestInterface1
{
    void Add(int num1, int num2);
}
interface ITestInterface2 : ITestInterface1
{
    void Sub(int num1, int num2);
}

public class ImplementationClass2 : ITestInterface2
{
    //Normal Implementation of Interface Method
    public void Add(int num1, int num2)
    {
        Console.WriteLine($"Sum of {num1} and {num2} is {num1 + num2}");
    }

    //Explicit Implementation of Interface Method
    void ITestInterface2.Sub(int num1, int num2)
    {
        Console.WriteLine($"Divison of {num1} and {num2} is {num1 - num2}");
    }
}
```

Normal Implementation using public access specifier

Explicit Implementation using Interface name

As you can see in the above code, the Add method is implemented using the public access specifier, and the Sub is implemented using the interface name. The Sub method belongs to Interface2, and hence, we prefix the Sub method with Interface2, followed by the dot operator. If you want to implement the Add method explicitly, you need to prefix the Add method with Interface1, as the Add method belongs to Interface1.

In this case, when we use the Interface name while implementing the interface method, we no longer need to use the public access specifier. The reason is that it is very clear that interface members are public, and hence, we aren't required to use the public access specifier. So, these are the two ways of implementing interface members in C#.

If the method is implemented using the public access specifier, then you can create the object and call it directly. But if the method is implemented using the interface name, then while calling the method, we need to typecast the object to the interface type, or you can create an interface reference and call the method. So, in our case, we call the Add method directly using obj1, but while calling the Sub method, we need to typecase the obj1 to Interface2 type as this is an instance of ImplementationClass, or you can call directly using reference variable obj2 as shown in the below image.

```

static void Main()
{
    ImplementationClass obj1 = new ImplementationClass();
    //Using obj1 we can call the Add method directly because
    //It is implemented using public access specifier
    obj1.Add(10, 20);

    //We need to typecast obj1 to ITestInterface1 to call the Sub
    //method because Sub method is implemented using Interface name
    ((ITestInterface1)obj1).Sub(100, 20);

    //We can call the method directly using the interface reference
    //Typecasting is not required in this case
    ITestInterface1 obj2 = new ImplementationClass();
    obj2.Add(200, 50);
    obj2.Sub(200, 50);

    Console.ReadKey();
}

```

Explicit Interface Implementation Example in C#

```

using System;
namespace AbstractClassMethods
{
    class Program
    {
        static void Main()
        {
            ImplementationClass obj1 = new ImplementationClass();
            //Using obj1 we can call the Add method directly because
            //It is implemented using public access specifier
            obj1.Add(10, 20);
            //We need to typecast obj1 to ITestInterface1 to call the Sub
            //method because Sub method is implemented using Interface name
            ((ITestInterface1)obj1).Sub(100, 20);
            //We can call the method directly using the interface reference
            //Typecasting is not required in this case
            ITestInterface1 obj2 = new ImplementationClass();
            obj2.Add(200, 50);
            obj2.Sub(200, 50);
            Console.ReadKey();
        }
    }
}

```

```

interface ITestInterface1
{
    void Add(int num1, int num2);
    void Sub(int num1, int num2);
}

public class ImplementationClass : ITestInterface1
{
    //Interface Method Implementation
    public void Add(int num1, int num2)
    {
        Console.WriteLine($"Sum of {num1} and {num2} is {num1 + num2}");
    }
    //This method purely belongs to ImplementationClass
    void ITestInterface1.Sub(int num1, int num2)
    {
        Console.WriteLine($"Divison of {num1} and {num2} is {num1 - num2}");
    }
}

```

Output:

```

Sum of 10 and 20 is 30
Divison of 100 and 20 is 80
Sum of 200 and 50 is 250
Divison of 200 and 50 is 150

```

When to use Interface in C#?

Interfaces in C# are a powerful feature that defines contracts or specifications that classes must adhere to. You should consider using interfaces in the following scenarios:

- **Defining a Common Contract:** To ensure that multiple classes provide a common set of methods, properties, events, or indexers, you can create an interface to define that contract. This is useful in situations where different classes need to exhibit similar behavior but may have different implementations.
- **Implementing Multiple Inheritance:** C# does not support multiple inheritance for classes (a class cannot inherit from multiple classes) but supports multiple interface implementation. If you need a class to inherit behavior or structure from multiple sources, you can use interfaces to achieve this. This allows you to share code among different classes without the complexities of multiple-class inheritance.
- **Enforcing a Specific Structure:** When you want to enforce a specific structure or a particular set of methods and properties for classes within your application or library, interfaces can help ensure that classes conform to that structure.
- **Implementing Polymorphism:** Interfaces are essential for achieving polymorphism in C#. When different classes implement the same interface, you can treat objects of these classes uniformly, making it easier to work with various objects in a polymorphic way. This is useful in scenarios where you want to work with objects at a higher level of abstraction.

- **Testing and Mocking:** Interfaces are valuable in unit testing scenarios. You can create interfaces for external dependencies or services and then create mock implementations of those interfaces for testing purposes. This enables you to isolate and test individual components of your codebase more effectively.
- **Collaboration in Teams:** Interfaces can help teams collaborate more effectively on large codebases. By defining clear interfaces, developers can work independently on different parts of a project, knowing that their code will integrate seamlessly as long as it adheres to the specified interfaces.
- **Code Reusability:** Interfaces promote code reusability. You can create a set of interfaces representing common functionalities in your application, and different classes can implement these interfaces. This way, you can reuse code across multiple classes without duplicating it.
- **Dependency Injection:** When using dependency injection, interfaces often define dependencies that can be injected into classes. This helps achieve loose coupling and makes it easier to switch implementations at runtime.

You should use interfaces in C# to define a contract, enforce a specific structure, achieve polymorphism, promote code reusability, or facilitate developer collaboration. They are an important part of the language for building maintainable and extensible software.

In the next article, I will discuss [Interface Interview Questions and Answers in C#](#) with Examples. In this article, I try to explain the **Interface in C#** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, questions, or comments about this Interface in C# with the Examples article.